SENSITIVE FILES

Bug Hunting Guide 2026

Exposed .env, .git, Backup Files, Config Leaks & More

A Practical Guide for Bug Bounty Hunters & Penetration Testers

Information Disclosure | Source Code Leaks | Credential Exposure | Backup Files | API Keys

Compiled from 2026 bug bounty methodologies, real-world write-ups, and OWASP standards

TABLE OF CONTENTS

1. WHY SENSITIVE FILE EXPOSURE MATTERS IN 2026

Sensitive file exposure is one of the most consistent and high-impact vulnerability classes in bug bounty hunting. In 2026, with the proliferation of cloud deployments, containerized applications, and rapid CI/CD pipelines, developers continue to accidentally leave critical files exposed on production servers.

Why this matters now more than ever:

- Cloud-First Development: Misconfigured S3 buckets, Azure blobs, and GCP storage frequently expose .env and config files
- Container Leaks: Docker layers and Kubernetes configs often contain embedded secrets that get exposed
- AI/ML APIs: Exposed API keys in .env files can lead to massive AI service abuse and financial drain
- Supply Chain Risk: .git exposure can reveal entire source code, leading to supply chain compromise
- Regulatory Pressure: GDPR, PCI-DSS, and HIPAA violations from credential exposure carry heavy fines
- Automated Scanning Limitations: Scanners miss context - manual hunting finds the real gold

The mindset shift: Sensitive files are not just configuration files. They are attack multipliers. A single exposed .env file can provide database credentials, API keys, JWT secrets, and cloud service tokens - everything needed for full system compromise.

> In 2026, bug bounty programs are increasingly classifying sensitive file exposure as High or Critical severity when it contains production credentials, especially for cloud services (AWS, Azure, GCP) or payment processors (Stripe, PayPal).

2. THE TOP SENSITIVE FILES TO HUNT

This section covers the most valuable sensitive files to target during reconnaissance and testing. Each file type has specific discovery methods, exploitation techniques, and impact levels.

2.1 .env Files - The API Key Goldmine

.env files are configuration files used by frameworks like Laravel, Symfony, Django, Flask, and Node.js to store environment variables. They almost always contain sensitive data.

What .env files typically contain:

- Database credentials (DB_HOST, DB_USER, DB_PASSWORD)
- API keys (STRIPE_KEY, SENDGRID_KEY, AWS_ACCESS_KEY_ID)
- JWT secrets and encryption keys (APP_KEY, SECRET_KEY)
- Mail server credentials (MAIL_HOST, MAIL_PASSWORD)
- Cloud service tokens (AWS, Azure, GCP configurations)
- Third-party service credentials (Slack, Twilio, Firebase)

Common .env File Locations:

```
/.env | /.env.local | /.env.production | /.env.development | /.env.backup | /api/.env | /app/.env | /config/.env | /backend/.env | /laravel/.env
```

Steps to Reproduce:

1. During reconnaissance, test these paths directly in browser or via curl

```
curl -I https://target.com/.env | curl -I https://target.com/api/.env | curl -I https://target.com/config/.env
```

2. Check for HTTP 200 responses (not 404 or 403)
3. If accessible, download and analyze the content
4. Look for patterns: KEY=, SECRET=, PASSWORD=, TOKEN=
5. Validate credentials by attempting connection to referenced services

Full credential compromise. An exposed .env file with AWS keys can lead to complete cloud environment takeover, data exfiltration, and lateral movement. With Stripe keys, attackers can process fraudulent transactions.

NEVER use exposed credentials to access customer data. Only validate that credentials are valid (e.g., test AWS key with aws sts get-caller-identity without accessing data). Using credentials to access data is illegal without explicit authorization.

2.2 .git Directory Exposure

The .git directory contains the entire version control history of a project. When exposed, attackers can download the complete source code, including deleted files, commit history, and sometimes embedded secrets.

What .git exposure reveals:

- Complete source code (all branches, all history)
- Deleted files that may contain hardcoded credentials
- Commit messages revealing internal architecture
- Developer names and email addresses (for social engineering)
- Internal API endpoints and business logic
- Configuration files and deployment scripts

Steps to Reproduce:

1. Check if /.git/ is accessible

```
curl https://target.com/.git/HEAD | curl https://target.com/.git/config | curl https://target.com/.git/index
```

2. If HEAD returns ref: refs/heads/main or similar - .git is exposed
3. Use git-dumper to download the entire repository

```
pip install git-dumper | git-dumper https://target.com/.git/ ./downloaded-repo | cd downloaded-repo && git log --all --oneline
```

4. Search for secrets in commit history

```
git log -p | grep -i password,secret,key,token | truffleHog .
```

5. Check for deleted files that may have contained credentials

```
git log --diff-filter=D --summary
```

Even if /.git/ returns 403, try /.git/config or /.git/HEAD individually. Some servers block directory listing but not individual file access. Also test /.gitignore - it often reveals file patterns that hint at other exposed files.

Complete source code disclosure. Attackers can find 0-day vulnerabilities in custom code, discover internal API endpoints, and find hardcoded credentials in deleted files. This is often rated High to Critical severity.

2.3 Backup & Config Files

Developers frequently create backup copies of configuration files, databases, and source code during maintenance. These backups are often forgotten on the server and left accessible.

Common Backup File Patterns:

File Pattern	Description	Severity
.bak, .backup, .old	Backup copies of config files	Medium-High
.sql, .sql.gz, .dump	Database dumps	Critical
.zip, .tar, .tar.gz	Archived source code	High
.swp, .swo	Vim swap files (may contain unsaved data)	Medium
.DS_Store	macOS metadata (reveals file structure)	Low-Medium
.htaccess.bak	Apache config backups	Medium
web.config.bak	IIS config backups	Medium-High
wp-config.php.bak	WordPress config backup	High
config.php~	Text editor backup (tilde suffix)	High
.env~	Environment file backup	Critical

Steps to Reproduce:

1. Fuzz for backup files using ffuf with a backup-specific wordlist

```
ffuf -u https://target.com/FUZZ -w
/usr/share/seclists/Discovery/Web-Content/backups.txt -mc 200
```

2. Manually test common backup suffixes on known config files

```
https://target.com/config.php.bak | https://target.com/.env.backup |
https://target.com/database.sql.gz | https://target.com/backup.zip
```

3. Check for editor swap files in common directories

```
https://target.com/.index.php.swp | https://target.com/config/.env.swp
```

4. Download and analyze any discovered files for credentials or sensitive data

Database dumps expose entire customer datasets. Source code archives reveal business logic and potential vulnerabilities. Config backups contain credentials for databases, APIs, and cloud services.

2.4 robots.txt & sitemap.xml

While robots.txt and sitemap.xml are not inherently sensitive, they often reveal hidden endpoints, admin panels, staging environments, and paths that developers intended to keep private.

What to Look For in robots.txt:

Disallow: /admin/ - Reveals admin panel location

Disallow: /api/internal/ - Hidden API endpoints

Disallow: /staging/ - Staging environment paths

Disallow: /backup/ - Backup directory locations

Disallow: /test/, /dev/, /debug/ - Development endpoints

Disallow: /.git/ - Confirms .git directory exists (even if blocked from crawlers)

What to Look For in sitemap.xml:

Hidden pages not linked from the main site

Admin or internal tool URLs

API documentation endpoints

Staging/test environment URLs

User profiles or content that should not be public

Steps to Reproduce:

1. Access https://target.com/robots.txt and https://target.com/sitemap.xml
2. Parse all Disallow entries and url entries
3. Test each discovered path for accessibility

```
curl -I https://target.com/admin/ | curl -I https://target.com/api/internal/ | curl -I https://target.com/staging/
```

4. Check if disallowed paths are actually accessible (robots.txt only blocks crawlers, not direct access)
5. Look for sitemap index files: /sitemap_index.xml, /sitemap1.xml

A robots.txt that says Disallow: /admin/ is essentially a signpost saying Admin panel here! Always test these paths. Many developers assume robots.txt provides security - it does not.

Information disclosure of hidden endpoints. While robots.txt exposure itself is typically Low severity, the discovered endpoints often lead to Medium-High or Critical findings (exposed admin panels, internal APIs, staging environments).

2.5 phpinfo() & Debug Pages

phpinfo() pages and framework debug pages are goldmines of information. They reveal server configuration, environment variables, loaded modules, and sometimes even request headers containing cookies or tokens.

What phpinfo() reveals:

- PHP version and loaded extensions (look for known CVEs)
- Server software and version (Apache/Nginx + version)
- Document root path (reveals file structure)
- Environment variables (may contain secrets)
- HTTP headers from current request (may leak cookies/tokens)
- Session save path (potential session file access)
- Open_basedir and disable_functions (reveals security posture)

Common phpinfo() Locations:

```
/phpinfo.php | /info.php | /php_info.php | /test.php | /i.php | /dashboard/phpinfo.php | /admin/phpinfo.php | /api/phpinfo.php
```

Framework Debug Pages:

- Django: /?debug=true, /debug/ (when DEBUG=True)
- Laravel: /_debugbar/ (if debugbar installed)
- Symfony: /_profiler/ (web profiler)
- Rails: /rails/info/routes, /rails/info/properties
- Spring Boot: /actuator/ endpoints (see Section 2.6)

Steps to Reproduce:

1. Test common phpinfo() paths directly

```
curl https://target.com/phpinfo.php | grep -i password,secret,key,token,AWS
```

2. Search the phpinfo() output for ENV, SERVER sections - these may contain environment variables
3. Check the HTTP Headers section for leaked authorization tokens or cookies
4. Note the PHP version and search for known CVEs affecting that version
5. For framework debug pages, test known paths and check if debug mode is enabled in production

Information disclosure ranging from Low (version info) to Critical (environment variables with credentials). Debug mode in production is almost always a High or Critical finding.

2.6 Swagger / OpenAPI Documentation

Swagger UI and OpenAPI documentation endpoints are incredibly valuable for attackers. They provide a complete map of the API, including endpoints, parameters, authentication requirements, and sometimes example responses with real data.

Common Swagger/OpenAPI Paths:

```
/swagger-ui.html | /swagger.json | /swagger.yaml | /api-docs | /api-docs/swagger.json |
/v2/api-docs | /v3/api-docs | /openapi.json | /openapi.yaml | /api/swagger.json |
/api/v1/swagger.json | /actuator/swagger-ui
```

What Swagger Docs Reveal:

- Complete API endpoint listing with HTTP methods
- Required and optional parameters for each endpoint
- Authentication mechanisms (Bearer tokens, API keys, OAuth)
- Request/response schemas (reveals data structures)
- Example requests that may contain real data
- Hidden or undocumented endpoints not visible in the main app
- Internal/admin endpoints that should not be public

Steps to Reproduce:

1. Test common Swagger paths on the target domain
2. If Swagger UI is accessible, explore all endpoints interactively
3. Download the swagger.json or openapi.json file for offline analysis
4. Look for endpoints with tags like admin, internal, debug, management
5. Test endpoints that require authentication - try accessing them without auth
6. Check example responses for real user data or credentials
7. Look for POST/PUT/DELETE endpoints that might allow unauthorized data modification

Some applications expose Swagger docs but have Try it out disabled. You can still download the JSON spec and use tools like Postman or curl to test the endpoints manually. Also check for /api-docs on subdomains - developers often forget to disable docs on staging or API subdomains.

Full API mapping enables targeted attacks on every endpoint. If admin endpoints are documented and accessible without proper authentication, this is a Critical finding. Even read-only access to Swagger docs is typically Medium-High severity.

3. AUTOMATED DISCOVERY TECHNIQUES

Manual testing is essential, but automation helps you cover more ground faster. Here are the most effective automated techniques for discovering sensitive files in 2026.

3.1 Nuclei Templates for Sensitive Files

Nuclei has extensive templates for sensitive file detection. Use these specific templates:

```
nuclei -u https://target.com -t http/exposures/configs/ | nuclei -u https://target.com
-t http/exposures/backups/ | nuclei -u https://target.com -t http/exposures/tokens/ |
nuclei -u https://target.com -t http/exposures/apis/
```

3.2 ffuf for File Fuzzing

Use ffuf with SecLists wordlists for comprehensive file discovery:

```
ffuf -u https://target.com/FUZZ -w
/usr/share/seclists/Discovery/Web-Content/raft-large-files.txt -w
/usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt -mc
200,301,302,401,403 -o results.json
```

3.3 Nikto for Known Sensitive Files

Nikto includes checks for common sensitive files and misconfigurations:

```
nikto -h https://target.com -C all
```

3.4 Wayback Machine for Historical Files

Historical snapshots may contain files that were later removed but still accessible:

```
waybackurls target.com | grep -E .(env|git|zip|sql|bak|config|yaml|json)$ | gau
target.com | grep -E .(env|git|zip|sql|bak)$
```

3.5 GitHub Dorking for Exposed Files

Search GitHub for exposed files related to the target:

```
filename:.env target.com | filename:config.php target.com | filename:id_rsa target.com |
filename:.htpasswd target.com
```

Combine multiple tools. Start with Nuclei for broad coverage, then use ffuf with custom wordlists for deep fuzzing. Always check Wayback Machine - developers often fix exposure by removing the link, but the file remains accessible at the old URL.

4. MANUAL TESTING METHODOLOGY

Automation finds the obvious. Manual testing finds the gold. Here is a systematic approach to manually hunting sensitive files in 2026.

Phase 1: Reconnaissance & Baseline

[ ] Identify the technology stack (Wappalyzer, response headers, favicon analysis)

[ ] Determine the framework (Laravel, Django, Rails, Spring Boot, etc.)

[ ] Check for common framework-specific paths:

Laravel: /.env, /storage/, /vendor/

Django: /admin/, /static/admin/

Rails: /rails/info/routes, /assets/

Spring Boot: /actuator/, /swagger-ui.html

WordPress: /wp-config.php, /wp-admin/

[ ] Establish baseline 404 response for comparison

Phase 2: Targeted File Testing

[ ] Test root-level sensitive files: /.env, /.git/HEAD, /robots.txt

[ ] Test common subdirectories: /api/, /admin/, /config/, /backup/

[ ] Test backup suffixes on known config files: .bak, .old, ~, .swp

[ ] Test for source code archives: .zip, .tar.gz, .rar

[ ] Test for database dumps: .sql, .sql.gz, .dump

[ ] Test for IDE/editor files: .idea/, .vscode/, .DS_Store

Phase 3: Response Analysis

[ ] Compare response lengths - different sizes suggest different page types

[ ] Check response headers for Content-Type - application/json on a config file is a strong indicator

[ ] Look for X-Powered-By and Server headers for technology hints

[ ] Check if 403 responses have different bodies than 404s - 403 means the file exists but is blocked

[ ] Test for bypass techniques on 403 files:

```
/config.php%00.jpg (null byte) | /config.php. (trailing dot) | /config.php/./ (path traversal) | /.env?anything=1 (query string bypass)
```

Phase 4: Content Validation

[ ] If a file is accessible, download and analyze it

[ ] Search for keywords: password, secret, key, token, aws, stripe, database

[ ] Validate any discovered credentials (without accessing data)

[ ] Check if credentials are for production or staging environments

[ ] Document the full path and access method for your report

When you find a 403 on a sensitive file, do not give up. Try bypass techniques, different HTTP methods (PUT, PATCH), and header variations. A 403 often means the file exists - finding a way to access it is the real challenge.

5. EXPLOITING EXPOSED FILES

Finding an exposed file is only the beginning. The real value comes from understanding how to exploit the information it contains. This section covers practical exploitation techniques for different file types.

5.1 Exploiting .env Files

When you find an exposed .env file with AWS credentials:

```
# Validate the credentials (read-only check) | aws configure set aws_access_key_id
AKIA... | aws configure set aws_secret_access_key ... | aws sts get-caller-identity | #
Check permissions without accessing data | aws iam list-attached-user-policies
--user-name found-user | aws s3 ls # Lists buckets only, no data access
```

5.2 Exploiting .git Exposure

After downloading the repository with git-dumper:

```
# Search for hardcoded credentials in all commits | git log --all -p | grep -i
password,secret,api_key | # Find deleted files that may have contained secrets | git log
--all --full-history -- .env | git show COMMIT_HASH:.env | # Use automated secret
scanning | truffleHog . | gitLeaks detect --source .
```

5.3 Exploiting Database Dumps

If you find a .sql dump file:

```
# Analyze structure without importing (grep for credentials) | grep -i
password,secret,token,api_key dump.sql | head -20 | # Check for user tables and password
hashes | grep -i INSERT INTO.*users,INSERT INTO.*password dump.sql | # Look for admin
accounts | grep -i admin,root,superuser dump.sql | head -10
```

5.4 Exploiting Swagger/OpenAPI Docs

When you find exposed API documentation:

```
# Download the spec file | curl https://target.com/swagger.json -o swagger.json | #
Import into Postman for interactive testing | # Or use swagger-codegen to generate
client code | # Test admin endpoints without authentication | curl
https://target.com/api/admin/users | curl https://target.com/api/internal/config
```

Only validate that credentials work - never use them to access customer data, modify systems, or perform destructive actions. Your report should demonstrate the vulnerability, not exploit it.

6. GOOGLE DORKS FOR SENSITIVE FILES

Google dorks are powerful for finding exposed sensitive files across the internet. Use these responsibly and only on targets you have permission to test.

.env File Dorks:

```
site:target.com inurl:.env | site:target.com DB_PASSWORD DB_HOST | site:target.com
APP_KEY APP_ENV=production | site:target.com filetype:env
```

.git Directory Dorks:

```
site:target.com inurl:.git | site:target.com index of .git | site:target.com
intitle:Index of /.git
```

Backup File Dorks:

```
site:target.com filetype:sql | site:target.com filetype:bak | site:target.com
filetype:zip inurl:backup | site:target.com index of backup
```

Config File Dorks:

```
site:target.com filetype:xml inurl:config | site:target.com filetype:json inurl:config |
site:target.com inurl:wp-config.php | site:target.com inurl:web.config
```

phpinfo() Dorks:

```
site:target.com inurl:phpinfo.php | site:target.com phpinfo() PHP Version |
site:target.com intitle:phpinfo()
```

Swagger/OpenAPI Dorks:

```
site:target.com inurl:swagger-ui.html | site:target.com inurl:api-docs | site:target.com
swagger api filetype:json
```

Combine dorks with the target's subdomains. Use site:*.target.com instead of just site:target.com to catch files on subdomains like api.target.com or staging.target.com.

7. REPORTING & CVSS SCORING

A well-written report is the difference between a duplicate and a payout. Here is how to report sensitive file exposure findings effectively in 2026.

Report Structure:

1. Title: Be specific - [High] Exposed .env file on api.target.com leaks AWS production credentials
2. Summary: One paragraph describing what you found and why it matters
3. Steps to Reproduce: Exact URLs, curl commands, and screenshots
4. Proof of Concept: Redacted screenshot showing the exposed data (blur actual credentials)
5. Impact: What an attacker could do with this information
6. CVSS Justification: Why you chose the severity rating
7. Remediation: Specific steps to fix the issue

CVSS 3.1 Scoring Guidelines for Sensitive Files:

Finding	Severity	CVSS Range	Justification
Exposed .env with staging credentials	Medium	4.0-6.9	Non-production, limited impact
Exposed .env with production DB credentials	High	7.0-8.9	Direct data access, no auth needed
Exposed .env with AWS root keys	Critical	9.0-10.0	Full cloud compromise possible
.git directory exposed (no credentials)	Medium	5.0-6.9	Source code disclosure, aids further attacks
.git exposed + credentials in history	High	7.0-8.9	Source + credential compromise
Database dump (.sql) accessible	Critical	9.0-10.0	Complete data breach
Backup archive (.zip) with source	High	7.0-8.9	Source code + potential secrets
robots.txt reveals admin path	Low	2.0-3.9	Information disclosure only
phpinfo() in production	Medium	5.0-6.9	Version disclosure, potential exploit path
phpinfo() with env vars + secrets	High	7.0-8.9	Direct credential exposure
Swagger docs without auth	Medium	4.0-6.9	API mapping, aids targeted attacks
Swagger docs + admin endpoints unprotected	High	7.0-8.9	Unauthorized admin API access

Always justify your CVSS score. Do not just say High - explain the Attack Vector (Network), Attack Complexity (Low), Privileges Required (None), and User Interaction (None). These factors make sensitive file exposure particularly dangerous.

Remediation Suggestions:

.env files: Move outside web root, use environment variable injection in production, never commit to version control

.git directories: Block access via web server config (nginx: location ~ /\.git { deny all; })

Backup files: Store outside web root, use .htaccess/nginx rules to block backup extensions

phpinfo(): Remove from production, disable debug mode, use environment-specific configs

Swagger: Require authentication for docs, disable in production, or restrict to internal IPs

General: Implement proper access controls, use web application firewalls, regularly audit file permissions

8. COMPLETE TESTING CHECKLIST

Use this comprehensive checklist during every reconnaissance and testing session. Check off each item to ensure thorough coverage.

RECONNAISSANCE PHASE

- [ ] Identify technology stack (Wappalyzer, headers, favicon)
- [ ] Enumerate subdomains (subfinder, assetfinder, amass)
- [ ] Check Wayback Machine for historical files (waybackurls, gau)
- [ ] Search GitHub for exposed files related to target
- [ ] Run Google dorks for sensitive files on target domain
- [ ] Check robots.txt and sitemap.xml on all subdomains

AUTOMATED SCANNING PHASE

- [ ] Run Nuclei with exposures templates (configs, backups, tokens, APIs)
- [ ] Run ffuf with large file/directory wordlists
- [ ] Run Nikto for known sensitive file checks
- [ ] Check Wayback Machine for removed but still accessible files
- [ ] Run GitHub dorking for target-related exposed files

MANUAL TESTING PHASE

- [ ] Test root-level sensitive files: /.env, /.git/HEAD, /robots.txt
- [ ] Test framework-specific paths based on identified stack
- [ ] Test backup suffixes on known config files
- [ ] Test for source code archives
- [ ] Test for database dumps
- [ ] Test for IDE/editor files
- [ ] Test 403 bypass techniques on blocked files

ANALYSIS PHASE

- [ ] Compare response lengths for anomalies
- [ ] Check Content-Type headers for file indicators
- [ ] Analyze discovered files for credentials and secrets
- [ ] Validate credentials without accessing data
- [ ] Document all findings with screenshots and curl commands

REPORTING PHASE

- [ ] Write clear, specific title
- [ ] Include exact reproduction steps
- [ ] Provide redacted PoC screenshots
- [ ] Justify CVSS score with specific metrics
- [ ] Suggest specific remediation steps
- [ ] Include impact assessment for business context

9. CONCLUSION

Sensitive file exposure remains one of the most reliable and high-impact vulnerability classes in 2026. While automated scanners can find the obvious exposures, the real value comes from systematic manual testing, creative path discovery, and thorough analysis of discovered files.

The key takeaways for hunting sensitive files:

- Never trust a clean 404. Hidden files often exist behind generic error pages.
- Use large wordlists. A 60,000-word list will miss what a 350,000-word list finds.
- Test bypass techniques on 403 responses. A blocked file is often an existing file.
- Check subdomains and staging environments. Developers often forget to secure non-production assets.
- Analyze file content deeply. A .env file is worthless if you do not extract and validate the credentials.
- Chain your findings. A robots.txt leak -> admin panel discovery -> default credentials -> full system access.
- Document everything. The quality of your report determines your payout.

Happy hunting. Go find those files.

Compiled from 2026 bug bounty methodologies, real-world write-ups, CVE analysis, and OWASP standards. For educational and authorized testing purposes only.